AfroRythm

Project Plan

Creator-first music streaming platform with MTN Mobile Money monetization

THIS DOCUMENT COVERS

- Product background and market gaps
- Implementation document
- Dashboard roles and tasks
- Feature backlog and delivery order
- Monetization implementation guide

Prepared for Ruby

Updated October 3, 2026

Contents

1. Product Background 4

Market Opportunity 4

Key Market Gaps 4

Full Feature List 4

Step-by-Step Build Process 6

Success Metrics 7

2. Implementation Document 8

1. Project Title 8

2. Project Overview 8

3. Problem Statement 8

4. Project Objectives 8

5. Scope of the System 8

6. Target Users 8

7. Core Features 8

8. Monetization Model 9

9. Premium Access Logic 9

10. Functional Requirements 9

11. Non-Functional Requirements 10

12. System Architecture 10

13. Main Modules 10

14. Database Design 11

15. Payment Flow 11

16. Premium Feature Enforcement 11

17. Content Moderation 12

18. Analytics and Reporting 12

19. Security Considerations 12

20. Development Phases 12

21. Step-by-Step Build Order 13

22. Recommended MVP Priorities 13

23. Expected Business Outcome 14

24. Risks and Mitigation 14

25. Conclusion 14

3. Dashboard Roles and Tasks . . . 15

1. Visitor Dashboard . . . 15
2. Listener / Fan Dashboard . . . 15
3. Artist Dashboard . . . 15
4. Premium Artist Dashboard . . . 16
5. Admin Dashboard . . . 16
6. Super Admin Dashboard . . . 17
7. Promoter / Curator Dashboard . . . 17
8. Payment Gateway Dashboard . . . 17
9. Notification Service Dashboard . . . 18
10. Analytics Dashboard . . . 18

Recommended dashboard structure by actor . . . 18

Best way to use this in your project . . . 19

4. Feature Backlog . . . 20

1. Product Goal . . . 20
2. Prioritization Method . . . 20
3. Backlog Legend . . . 20
4. Feature Backlog Table . . . 20
5. Must-Build Features First . . . 22
6. Why These Come First . . . 22
7. High-Value Features Next . . . 22
8. Later-Stage Features . . . 23
9. Recommended Dashboard Placement . . . 23
10. Suggested Delivery Order . . . 23
11. Feature Selection Rule . . . 24
12. Final Recommendation . . . 24

5. Monetization Implementation Guide . . . 25

5.1 The Big Picture . . . 25

5.2 Decide Your Plans First . . . 26

5.3 Build Order . . . 26

5.4 Going Live With Real MoMo . . . 31

5.5 Master Checklist . . . 32

5.6 Risks and How This Plan Handles Them . . . 32

6. Campay Payments Reference . . . 33

Sections 6.1 to 6.10, pages 33 to 40 (added Oct 5, 2026)

1. Product Background

Market context, the full feature list, the build process, and success metrics. Architecture, database design, and security are covered in the Implementation Document that follows.

Market Opportunity

The opportunity is strong because recorded music revenue in sub-Saharan Africa is growing while creator monetization problems remain unresolved. That creates room for a platform that combines discovery, direct support, and local payment methods for artists who are underserved by large streaming services.

Key Market Gaps

The market gaps this project should target are:

- Weak monetization for emerging artists.
- Poor visibility for young and local talent.
- Limited local payment integration on creator platforms.
- Low trust in payout transparency.
- Too much focus on large catalogs instead of talent development.
- Limited support for micro-payments and fan tipping.
- Weak promotion tools for unsigned artists.

Full Feature List

1. Artist onboarding

Artists register with profile details, genre, location, and social links. This is important because the platform needs identity, creator segmentation, and a basis for payouts.

2. Music upload system

Artists upload tracks, covers, metadata, and release details. This is the core supply-side feature and should support drafts, publishing, editing, and takedown requests.

3. Artist profile pages

Each artist gets a public profile with music catalog, followers, earnings summary, and featured releases. This helps discovery and makes the platform feel creator-owned.

4. Streaming and playback

Users can play music, pause, skip, queue, and browse tracks. The app should support low-data playback because mobile bandwidth and affordability matter for the target audience.

5. Discovery feed

A personalized discovery feed should surface new artists, trending songs, and local recommendations. This directly targets the market gap where unsigned talent is hard to find.

6. Follow and fan engagement

Fans can follow artists, like songs, comment, and share releases. Engagement features help artists grow organically and give the app a community layer beyond streaming.

7. MTN Mobile Money payments

Users can pay with MTN Mobile Money for tips, subscriptions, premium content, or downloads. MTN's API supports payment-related products including collection and disbursement, and the developer portal documents sandbox and production-style integration paths.

8. Wallet and earnings dashboard

Artists should see payments received, pending withdrawals, transaction history, and payout status. This is critical because payout transparency is one of the strongest differentiators you can build.

9. Payout system

The platform should allow creators to withdraw earnings to MTN Mobile Money wallets. MTN's disbursement support makes this technically realistic, subject to the operational setup and compliance requirements of the specific market.

10. Tipping and micro-support

Fans should be able to send small payments to artists as direct support. This is a strong fit for emerging creators because it monetizes even small communities.

11. Subscriptions and memberships

Artists can offer paid fan access to exclusive tracks, early releases, or premium content. This gives creators more than one revenue stream and avoids dependence on streaming alone.

12. Promotions and featured placements

Artists can pay for boosted visibility or featured slots. This is useful for early-stage monetization and can help creators get discovered faster.

13. Playlist and curation tools

Admins or curators can build playlists by genre, mood, region, or rising talent category. Curation helps solve the visibility problem that large platforms often handle poorly for small artists.

14. Notifications

Users get alerts for uploads, tips, follows, comments, and payouts. This improves retention and helps artists respond quickly to fan activity.

15. Admin moderation panel

Admins can approve tracks, reject copyrighted content, handle abuse, and manage disputes. This is necessary because copyright and payout trust are major risks in music platforms.

16. Analytics dashboard

Artists and admins can track streams, engagement, revenue, top songs, and conversion rates. Analytics help creators understand what content works and how to grow.

17. Copyright and takedown handling

The system should allow reporting, review, and removal of disputed tracks. This is a core risk-control feature for any music platform.

Step-by-Step Build Process

This is the detailed version of the build sequence summarized in section 21 of the Implementation Document.

Step 1: Define the MVP scope

Pick one country, one target artist segment, and one primary payment flow. MTN's API guidance shows that API support varies by market and product, so narrowing scope reduces integration risk.

Step 2: Build authentication and profiles

Create user sign-up, login, artist account types, and profile pages. This gives structure to the platform before music and payments are added.

Step 3: Build upload and storage

Implement music upload, file validation, artwork upload, metadata fields, and track publishing. This is the content foundation of the app.

Step 4: Build playback and discovery

Add the music player, search, categories, trending content, and artist discovery feed. Without discovery, artists will not get the exposure that makes the platform valuable.

Step 5: Integrate MTN Mobile Money collection

Implement payment collection for tipping, subscriptions, and premium content. MTN's MoMo API documents collection and request-to-pay style flows for receiving payments.

Step 6: Build earnings tracking

Show a dashboard for money received, pending balance, completed transactions, and payout history. This is one of your main differentiators because artists need trust and transparency.

Step 7: Add payouts

Enable withdrawals from the platform wallet to MTN Mobile Money. MTN's disbursement support makes creator payouts practical once compliance and account setup are complete.

Step 8: Add moderation and compliance

Build copyright reporting, admin review, takedown tools, and content approval. Music platforms need this early because legal and trust risks grow quickly.

Step 9: Add fan monetization tools

Introduce tipping, subscriptions, paid releases, and featured artist support. These features strengthen the platform's business model beyond simple streaming.

Step 10: Add analytics and growth tools

Add performance dashboards, referral tracking, trend insights, and campaign tools. Once content and payments are working, analytics will help optimize growth and revenue.

Success Metrics

- Number of artist signups.
- Number of uploaded tracks.
- Monthly active listeners.
- Payment conversion rate.
- Total creator earnings.
- Withdrawal completion rate.
- Artist retention.
- Track engagement rate.

2. Implementation Document

1. Project Title

Music Streaming and Creator Monetization Platform with MTN Mobile Money. The platform is designed for young and emerging artists to upload music, get discovered, and earn money directly from fans.

2. Project Overview

This project is a creator-first music platform that combines streaming, artist promotion, and digital payments. Its main purpose is to solve the gap between artist visibility and artist income by giving young talent a place to publish content and monetize it through direct fan support.

3. Problem Statement

Many emerging artists struggle to get visibility on major streaming platforms, and even when they get plays, earnings are often too small or too indirect. The platform solves this by adding local payment support, creator earnings dashboards, and premium tools that help artists earn from fans more directly.

4. Project Objectives

- Allow artists to upload and manage music.
- Let fans discover, stream, and support artists.
- Integrate MTN Mobile Money for payments and payouts.
- Offer premium creator tools and fan membership plans.
- Provide analytics, promotion, and monetization tools.

5. Scope of the System

The first version should launch in one country or a small MTN-supported region, not across all markets at once. The initial scope should focus on artist uploads, streaming, discovery, payment collection, creator payouts, and premium access control.

6. Target Users

- Young and emerging artists.
- Fans who support local music.
- Admins who moderate content and manage payouts.
- Promoters or curators who feature new talent.

7. Core Features

7.1 Free user features

- Register and log in.

- Browse and stream music.
- Follow artists.
- Share tracks.
- View limited artist pages.

7.2 Artist features

- Create an artist profile.
- Upload music and artwork.
- Edit metadata and releases.
- View basic analytics.
- Receive direct fan tips.
- Request payouts.

7.3 Premium artist features

- Unlimited or expanded uploads.
- Advanced analytics.
- Featured promotion slots.
- Custom branding.
- Faster payout visibility.
- Campaign tools and audience insights.

7.4 Premium fan features

- Early access to tracks.
- Exclusive content.
- Ad-free listening.
- Fan badges.
- Membership communities.

8. Monetization Model

The app owner earns from commissions on payments, premium artist subscriptions, fan subscriptions, promotion credits, and possibly advertising later. The strongest early revenue streams are transaction commission, premium creator accounts, and paid promotion features.

9. Premium Access Logic

Premium access must be enforced by the backend, not only by the front end. A user pays, the payment is confirmed through the MTN Mobile Money flow, the backend updates the subscription status, and the app unlocks the purchased features based on entitlement checks.

10. Functional Requirements

- User authentication.

- Role-based access control.
- Artist onboarding.
- Track upload and management.
- Playback and discovery.
- Payments and wallet integration.
- Subscription management.
- Payout processing.
- Notifications.
- Admin moderation.
- Analytics reporting.

11. Non-Functional Requirements

- Fast loading on mobile networks.
- Secure payment handling.
- Scalable architecture.
- Reliable uptime.
- Content moderation support.
- Responsive UI.
- Strong data privacy and access control.

12. System Architecture

The system should use a client-server model with a React front end, a Node.js/Express backend, a database for users and transactions, and cloud storage for audio files. MTN Mobile Money should be connected through server-side API calls, with payment confirmation updating subscription and wallet records.

13. Main Modules

- Authentication module.
- Artist profile module.
- Upload module.
- Streaming module.
- Discovery module.
- Payment module.
- Subscription module.
- Wallet and payout module.
- Admin module.
- Analytics module.
- Notification module.

14. Database Design

Core tables

- Users.
- ArtistProfiles.
- Tracks.
- Albums.
- Playlists.
- Streams.
- Payments.
- Subscriptions.
- WalletTransactions.
- Withdrawals.
- Promotions.
- Notifications.
- Reports.

Important subscription fields

- userId.
- planName.
- status.
- startDate.
- endDate.
- paymentReference.
- entitlementLevel.

15. Payment Flow

1. User selects a plan or support action.
2. App sends payment request through MTN Mobile Money.
3. MTN returns transaction status.
4. Backend verifies success.
5. Subscription or wallet balance is updated.
6. Premium access is unlocked automatically.

16. Premium Feature Enforcement

Each premium action must call an authorization check before execution. For example, the analytics screen, upload-limit endpoint, promotion endpoint, and payout endpoint should all verify whether the user plan includes those rights.

17. Content Moderation

The admin panel should allow track review, copyright reports, takedowns, and abuse handling. This is necessary because music platforms face legal and trust issues if copyrighted or harmful content is not controlled.

18. Analytics and Reporting

Artists should be able to see plays, likes, shares, tips, subscriptions, earnings, and conversion rate. Admins should see total traffic, top artists, payment totals, and plan performance.

19. Security Considerations

- Use JWT or secure session auth.
- Validate payment callbacks on the server.
- Protect premium APIs with entitlement checks.
- Encrypt sensitive data.
- Log payment and payout events.
- Apply rate limiting and abuse protection.

20. Development Phases

Phase 1: MVP foundation

- Authentication.
- Artist onboarding.
- Track upload.
- Streaming.
- Discovery.
- Basic admin moderation.

Phase 2: Monetization

- MTN Mobile Money integration.
- Tips.
- Subscriptions.
- Premium plan checks.
- Earnings dashboard.
- Payouts.

Phase 3: Growth

- Analytics.
- Featured promotions.
- Fan communities.
- Notifications.

- Referral system.
- Expanded curation tools.

Phase 4: Scale

- Multi-country support.
- Better recommendation engine.
- Ad system.
- Partner portals.
- Advanced fraud and copyright controls.

21. Step-by-Step Build Order

1. Define plans and entitlements.
2. Build authentication and roles.
3. Create artist profiles.
4. Build upload and storage.
5. Build playback and discovery.
6. Add payment collection.
7. Confirm payment and activate plans.
8. Add premium route protection.
9. Build wallet and payout logic.
10. Add analytics and admin controls.
11. Add promotions and subscriptions.
12. Test, fix, and deploy.

22. Recommended MVP Priorities

Highest priority

- Login and registration.
- Artist account creation.
- Music upload.
- Streaming player.
- Discovery feed.
- MTN payment integration.
- Subscription verification.

Medium priority

- Tipping.
- Artist wallet.
- Payout requests.
- Basic analytics.

- Content moderation.

Later priority

- Paid promotions.
- Fan clubs.
- Advanced analytics.
- Ads.
- Multi-country rollout.

23. Expected Business Outcome

If executed well, the platform can become a local creator economy product that earns from payments, premium tools, and artist growth services. The strongest positioning is not “a Spotify clone,” but a platform that helps young talent grow and earn directly.

24. Risks and Mitigation

- Copyright risk: add moderation and takedown tools.
- Low adoption: launch with a narrow niche and country.
- Payment failure: verify callbacks and retries.
- Weak revenue: use multiple monetization streams.
- User bypass of premium: enforce backend entitlement checks.

25. Conclusion

This project is feasible, commercially interesting, and technically realistic if you build it in stages and keep the first release focused. The most important idea is to combine discovery, creator monetization, and reliable local payments into one clear product.

3. Dashboard Roles and Tasks

Below is a well-structured dashboard document for your platform. It lists each actor, the roles assigned to that actor, and the tasks they perform from their dashboard when navigating those roles. The structure follows common music-platform and admin-dashboard patterns used in streaming and creator tools.

1. Visitor Dashboard

A visitor is a user who has not signed up yet. This dashboard is mostly public and acts as the entry point into the platform.

Roles and tasks

- Browse landing pages.
- View featured artists and trending music.
- Search public artist profiles.
- Read about premium plans.
- Create an account or sign in.

2. Listener / Fan Dashboard

This dashboard is for registered fans who listen to music and support artists. It should be simple, fast, and centered on discovery, engagement, and payments.

Roles and tasks

- Stream music.
- Search songs, albums, and artists.
- Follow or unfollow artists.
- Like, comment, and share tracks.
- Create and manage playlists.
- Tip artists using MTN Mobile Money.
- Subscribe to premium fan plans.
- Access exclusive or early-release content.
- View listening history and saved items.
- Manage account settings and notifications.

3. Artist Dashboard

This dashboard is for creators who upload and manage their music. It should focus on publishing, revenue, audience growth, and promotion.

Roles and tasks

- Create and edit artist profile.
- Upload songs, albums, and cover art.

- Add metadata such as title, genre, description, release date, and tags.
- Edit or remove uploaded content.
- View stream counts and fan engagement.
- Track earnings and payout status.
- Request withdrawal to MTN Mobile Money.
- Launch paid promotions or featured placements.
- Read fan feedback and comments.
- View audience insights and analytics.
- Manage subscription plan and upgrade options.

4. Premium Artist Dashboard

This dashboard is for artists who have purchased a paid plan. It contains everything in the normal artist dashboard, plus locked premium tools that improve growth and monetization.

Roles and tasks

- Access advanced analytics.
- View detailed revenue breakdowns.
- Use unlimited or expanded upload limits.
- Promote tracks from the dashboard.
- Unlock branded profile customization.
- See fan conversion and subscription data.
- Access priority support.
- Manage exclusive premium campaigns.
- Track performance by region and audience segment.

5. Admin Dashboard

The admin dashboard is the control center for the whole platform. It is used to supervise users, content, monetization, and system health.

Roles and tasks

- View platform overview statistics.
- Manage users and artist accounts.
- Approve or reject uploaded content.
- Handle reports and copyright complaints.
- Moderate comments and abuse cases.
- Review payment issues and failed transactions.
- Manage featured music and homepage content.
- Track revenue and subscription activity.
- Send announcements and platform notifications.
- Monitor system performance and logs.

6. Super Admin Dashboard

This is the highest-level control panel. It is for the platform owner or top system manager and includes sensitive administrative and configuration tasks.

Roles and tasks

- Create and manage admin accounts.
- Define user roles and permissions.
- Configure monetization rules.
- Set subscription prices and plan features.
- Control payment gateway settings.
- Manage platform-wide policies.
- View audit logs and security records.
- Control feature flags and system modules.
- Manage integrations and global settings.
- Oversee strategic platform performance.

7. Promoter / Curator Dashboard

This dashboard is for people who help push music into the right audience. It focuses on discovery, curation, and promotional support.

Roles and tasks

- Create playlists and collections.
- Feature selected artists.
- Recommend songs for trending sections.
- Run promotional campaigns.
- Organize talent showcases.
- Track campaign performance.
- Monitor which songs are gaining traction.
- Support artist visibility strategies.

8. Payment Gateway Dashboard

This is the external payment actor, mainly MTN Mobile Money, but the platform can also have an internal payment control area. It handles the financial side of user purchases and creator payouts.

Roles and tasks

- Process user payments.
- Confirm successful transactions.
- Support collections for subscriptions and tips.
- Support disbursements for artist payouts.
- Return payment status to the platform.

- Trigger webhook or callback responses.
- Help validate failed or pending transactions.

9. Notification Service Dashboard

This can be a system panel or internal service area for platform alerts. It helps the app keep users informed and engaged.

Roles and tasks

- Send upload confirmations.
- Notify artists of tips and subscriptions.
- Alert users about new releases.
- Send payout notifications.
- Broadcast admin announcements.
- Notify about failed payments or renewals.
- Deliver engagement alerts such as likes and comments.

10. Analytics Dashboard

This dashboard is used by artists, admins, and super admins to understand performance. It should present data visually and support decision-making.

Roles and tasks

- Show stream statistics.
- Display revenue charts.
- Show subscriber growth.
- Compare track performance.
- Track fan engagement.
- Monitor conversion rates.
- Analyze top-performing content.
- Review geographic and demographic insights.

Recommended dashboard structure by actor

Listener dashboard sections

- Home.
- Discover.
- Search.
- Library.
- Playlists.
- Support artists.
- Subscriptions.
- Settings.

Artist dashboard sections

- Overview.
- Upload music.
- My releases.
- Earnings.
- Payouts.
- Promotions.
- Analytics.
- Settings.

Admin dashboard sections

- Overview.
- Users.
- Content moderation.
- Payments.
- Reports.
- Analytics.
- Settings.
- Audit logs.

Best way to use this in your project

You should treat each dashboard as a role-based workspace. After login, the system should detect the user role and redirect them to the correct dashboard, then only show the tasks allowed for that role. That keeps the app clean, secure, and easy to use.

4. Feature Backlog

Below is a structured feature backlog for your music platform, written to help you plan, prioritize, and build in a practical order. It focuses on user demand, market gaps, monetization, and implementation effort, which are the same factors product teams use when ranking backlog items.

1. Product Goal

Build a creator-first music platform where young artists can upload music, grow an audience, and earn money directly through MTN Mobile Money. The backlog is organized to maximize early value, reduce risk, and support a realistic MVP.

2. Prioritization Method

This backlog uses a simple value-vs-effort approach combined with MVP thinking. Features that solve urgent user pain, generate revenue, or reduce platform risk are ranked first, while advanced or optional items are ranked later.

3. Backlog Legend

- Priority 1 = Must build first.
- Priority 2 = Should build after MVP.
- Priority 3 = Nice to have later.
- Effort: Low, Medium, High.
- Value: High, Medium, Low.

4. Feature Backlog Table

Priority 1 items are built first, Priority 2 after the MVP, and Priority 3 later. Items are grouped by business value and implementation effort.

ID	Feature	Main Actor	User Need / Market Gap	Priority	Value	Effort	Why it matters
F01	User registration and login	Visitor, Listener, Artist	Basic access and identity	1	High	Low	Required for all dashboards and role-based access.
F02	Role-based dashboard routing	All users	Show correct workspace per role	1	High	Medium	Prevents confusion and supports secure access.
F03	Artist profile creation	Artist	Visibility and identity	1	High	Low	Foundation for uploads and discovery.
F04	Music upload and publishing	Artist	Content creation	1	High	Medium	Core supply-side feature.
F05	Music playback / streaming	Listener	Listening and discovery	1	High	Medium	Core user value.

ID	Feature	Main Actor	User Need / Market Gap	Priority	Value	Effort	Why it matters
F06	Search and discovery feed	Listener, Visitor	Find talent easily	1	High	Medium	Solves weak discovery for young artists.
F07	MTN Mobile Money payment collection	Listener, Artist	Local payments	1	High	High	Needed for monetization and creator support.
F08	Subscription / plan purchase flow	Listener, Artist	Upgrade access	1	High	Medium	Enables premium revenue.
F09	Backend entitlement checks	System	Lock premium access	1	High	Medium	Prevents free access to paid features.
F10	Basic artist earnings dashboard	Artist	Earnings transparency	1	High	Medium	Builds trust with creators.
F11	Admin moderation panel	Admin	Content safety	1	High	Medium	Needed for copyright and abuse control.
F12	Tipping / micro-support	Listener	Direct support to artists	2	High	Medium	Strong monetization gap opportunity.
F13	Payout request to MTN wallet	Artist	Withdraw earnings	2	High	High	Completes the earning loop.
F14	Fan subscriptions / memberships	Listener	Exclusive access	2	High	Medium	Recurring revenue and superfans.
F15	Paid exclusive content	Artist, Listener	Unlock special tracks	2	High	Medium	Monetization and fan loyalty.
F16	Featured promotions / boosts	Artist	Better visibility	2	High	Medium	Artists pay for exposure.
F17	Follow / unfollow artists	Listener	Stay connected	2	Medium	Low	Increases retention and engagement.
F18	Likes, comments, shares	Listener	Social engagement	2	Medium	Low	Supports discovery and community.
F19	Notifications system	All users	Engagement alerts	2	Medium	Medium	Improves retention.
F20	Basic analytics for artists	Artist	Track growth	2	High	Medium	Helps creators improve performance.
F21	Playlist creation	Listener	Organize music	2	Medium	Medium	Common user expectation.
F22	Curated playlists by genre/city	Curator, Admin	Discovery and promotion	2	High	Medium	Helps surface young talent.
F23	Artist story / behind-the-track cards	Artist, Listener	Authenticity and context	3	Medium	Low	Matches what users want more of.
F24	Live streaming / fan Q&A	Artist, Listener	Direct artist connection	3	High	High	Strong community feature, but heavier build.
F25	Fan clubs / private communities	Artist, Listener	Superfan engagement	3	High	High	Good for recurring revenue.

ID	Feature	Main Actor	User Need / Market Gap	Priority	Value	Effort	Why it matters
F26	Advanced analytics	Premium Artist	Deep performance insight	3	High	High	Valuable, but not needed for MVP.
F27	Recommendation engine	System	Personalized discovery	3	High	High	Useful after more user data exists.
F28	Merch or ticket sales	Artist, Listener	In-app commerce	3	Medium	High	Great later revenue stream.
F29	Copyright reporting and takedown workflow	Admin	Legal protection	2	High	Medium	Important for platform safety.
F30	Referral system	All users	Growth and acquisition	3	Medium	Medium	Helps user growth later.

5. Must-Build Features First

These are the highest priority because they form the core product:

- Registration and login.
- Role-based dashboard access.
- Artist profiles.
- Upload and streaming.
- Search and discovery.
- MTN Mobile Money payments.
- Premium access control.
- Basic earnings dashboard.
- Admin moderation.

6. Why These Come First

These features solve the biggest market gaps: discovery, monetization, and trust. If you build them first, you can test whether artists upload, fans listen, and payments actually happen before spending time on advanced features.

7. High-Value Features Next

After the MVP, the next most useful features are tipping, subscriptions, paid content, featured promotion, notifications, and analytics. These features create the first real revenue engine for the platform and improve retention.

8. Later-Stage Features

Features such as live streaming, fan clubs, recommendation engines, and merch sales are powerful, but they need more traffic, more trust, and more infrastructure. They should be added after the app proves strong usage and monetization.

9. Recommended Dashboard Placement

- Visitor dashboard: F01, F06, F30.
- Listener dashboard: F05, F06, F12, F14, F17, F18, F19, F21.
- Artist dashboard: F03, F04, F10, F13, F15, F16, F20, F23, F24, F26, F28.
- Admin dashboard: F11, F22, F29.
- Super admin dashboard: plan control, access control, payment settings, system config.
- Payment service: F07, F08, F09, F13.

10. Suggested Delivery Order

Sprint 1

- Authentication.
- Role routing.
- Artist profile.
- Basic UI shells.

Sprint 2

- Upload system.
- Streaming player.
- Search and discovery.

Sprint 3

- MTN payment integration.
- Subscription plans.
- Entitlement protection.

Sprint 4

- Earnings dashboard.
- Admin moderation.
- Copyright reporting.

Sprint 5

- Tipping.
- Payouts.
- Notifications.

Sprint 6

- Promotions.
- Analytics.
- Fan memberships.

11. Feature Selection Rule

If a feature does one of these, move it up the backlog:

- Helps users discover talent.
- Helps artists earn money.
- Builds trust and safety.
- Increases retention.
- Improves conversion to paid plans.

12. Final Recommendation

Do not overload the first version with too many advanced social or AI features. Build the money loop first: artist uploads music, fans discover it, fans pay, artists get paid, and the app owner earns commission and premium fees.

5. Monetization Implementation Guide

Step-by-step build instructions for premium plans, MoMo payments, wallet, payouts and promotions, written for a beginner.

> **How this connects to the rest of the plan.** Chapters 1 to 4 describe what the system must do. This chapter explains how to build the monetization part, step by step. Where earlier chapters are brief, for example the Implementation Document lists only a few subscription fields, this chapter gives the detail: the plans and prices, the payment flow, the wallet, and the code.

What changed	Why
Role and plan are separate fields	An artist can be on any plan, and one field cannot hold both
Payment activation waits for gateway confirmation	The frontend must never decide a payment succeeded
Wallet uses a ledger instead of one balance number	Gives artists the payout transparency promised in the Product Background
Gateway adapter built first	Switch from simulated to real MoMo by changing one setting

5.1 The Big Picture

How AfroRythm makes money

1. **Subscriptions.** Fans and artists pay monthly for extra features.
2. **Commission on tips.** A fan tips an artist 1,000 XAF, and the platform keeps a small share (for example 10%).
3. **Promotion credits.** Artists buy credits once and spend them to boost songs.

The one rule that matters most

> **Premium access must be enforced on the backend, never only in React.**

Hiding a button in the UI is not security. Anyone can open browser dev tools or call your API URL directly. So the **server** must check "is this user allowed?" before every premium action.

The money loop

```
User taps "Upgrade" → Frontend asks Backend to start payment
→ Backend asks the Payment Gateway (MoMo) → User approves on phone
→ Gateway confirms to Backend → Backend activates the plan
→ Frontend refreshes and unlocks features
```

The frontend **never** decides that a payment succeeded. Only the backend does, after the gateway confirms.

Words you'll see

Term	Plain meaning
Gateway	The company that moves money for you (Campay, Fapshi, MTN). Your code talks to it.
Collection	Money coming *in* (fan pays you).
Disbursement	Money going *out* (you pay an artist).
Webhook	A URL on your server that the gateway calls to say "payment succeeded".
Entitlement	One specific thing a plan allows, like "advanced analytics".
Middleware	A function that runs *before* your route and can block the request.
Idempotency	Doing the same request twice has the same effect as once (prevents double charging).
Sandbox	A fake test version of the gateway where no real money moves.

5.2 Decide Your Plans First

Do this on paper before writing code. Changing it later is painful.

Plan	For	Price (starting idea)	What it unlocks
Free	Everyone	0 XAF	Listen, follow, share. Artists: limited uploads, basic stats
Fan Support	Listeners	~500 to 1,000 XAF/month	Ad-free, premium songs, early releases, badge
Artist Starter	New artists	~1,500 to 2,500 XAF/month	More uploads, tipping, simple analytics
Artist Pro	Serious artists	~5,000 XAF/month	High/unlimited uploads, advanced analytics, promotions, custom branding
Promotion credits	Artists	One-time (e.g. 1,000 XAF = 10 credits)	Boost a song

These prices are only starting guesses. Launch simple, then adjust after seeing who actually pays.

5.3 Build Order

Step 1: Fix the User model (about 30 minutes)

The problem: your `subscriptionStatus` field mixes two different ideas:

- **Role** = who you are (fan, artist, admin)
- **Plan** = what you've paid for (free, premium, pro)

An artist can be on the Free *or* Pro plan, so one field can't hold both. Also, your code uses `'vip'` but the enum doesn't allow it, so VIP saves fail silently.

What to do in `User.ts`:

```
role: { type: String, enum: ['fan', 'artist', 'admin'], default: 'fan' },
plan: {
  type: String,
  enum: ['free', 'fan_support', 'artist_starter', 'artist_pro'],
  default: 'free'
},
planExpiresAt: { type: Date, default: null },
```

Important: you already have users in your database. Existing users won't have `plan` yet, so write a one-time script that sets `plan: 'free'` for everyone and copies old `artist` statuses into `role`. Back up your database first (export it) before running anything.

Step 2: Create plans and entitlements (about 1 hour)

Instead of writing `if (plan === 'pro' || plan === 'vip' ...)` everywhere, keep **one list** of what each plan allows.

Create `src/config/plans.ts`:

```
export const PLANS = {
  free:           { price: 0,    maxUploads: 5,  features: [] },
  fan_support:    { price: 1000, maxUploads: 0,  features: ['premium_songs', 'ad_free'] },
  artist_starter: { price: 2000, maxUploads: 30, features: ['tipping', 'basic_analytics'] },
  artist_pro:     { price: 5000, maxUploads: 500, features: ['tipping', 'basic_analytics',
                    'advanced_analytics', 'promotions', 'custom_branding'] },
} as const;

export function hasFeature(plan: keyof typeof PLANS, feature: string) {
  return (PLANS[plan].features as readonly string[]).includes(feature);
}
```

Now the rest of your code asks `hasFeature(user.plan, 'advanced_analytics')`. To change what Pro includes, you edit one file.

Also: your existing `Plan` database model can stay for admin editing later, but start with this config file because it's simpler.

Step 3: Build the payment gateway "adapter" (about 2 hours)

Why: right now your payments are simulated. Later you'll connect real MoMo. If your routes call the simulator directly, you'd rewrite everything. Instead, make a standard shape ("interface") that both the fake and real gateway follow. Then switching is one setting.

1. Create `src/services/paymentGateway.ts` containing the interface (the `InitiateParams`, `InitiateResult`, `VerifyResult`, `DisburseParams`, `DisburseResult` types and the `PaymentGateway` interface from the original plan).
2. Rename `momoService.ts` to `simulatedGateway.ts` and make it `implements PaymentGateway`. TypeScript will underline anything missing, so fix those.
3. Create `src/services/gatewayFactory.ts`:

```
export function getPaymentGateway(): PaymentGateway {
  switch (process.env.PAYMENT_GATEWAY) {
    // case 'campay': return new CampayGateway();  // added in Step 11
    default: return new SimulatedGateway();
  }
}
```

4. In `.env` add `PAYMENT_GATEWAY=simulated`.
5. In `routes/momo.ts`, replace every direct `momoService` call with `getPaymentGateway()`.

Test: run a simulated payment and confirm it behaves exactly as before.

Step 4: Make the payment flow safe (about 3 hours)

This is the heart of monetization. Follow it carefully.

4a. Record every attempt first. Before talking to the gateway, create a `Payment` document with `status: 'pending'`. Add these fields to your Payment model: `gatewayTransactionId`, `phone`, `purpose` (`'subscription' | 'tip' | 'credits'`), `planName`.

4b. Prevent double charges. The frontend generates a unique `idempotencyKey` (any random string, e.g. `crypto.randomUUID()`) per click. The backend refuses to create a second payment with the same key.

4c. Never trust the price from the frontend. The frontend sends only `planId`. The **backend** looks up the price in `PLANS`. Otherwise someone could send "price: 1" and get Pro for 1 XAF.

4d. Activate only after confirmation. The sequence:

1. `POST /api/payments/initiate` creates the pending payment and calls the gateway. The user gets a prompt on their phone.
2. The gateway calls your webhook `POST /api/momo/webhook` when done. Your code checks the signature is genuine, finds the Payment by transaction ID, and if status is successful, sets it to `completed` and activates the plan (Step 5).
3. As a **backup**, the frontend polls `GET /api/payments/:id/status` every 3 seconds for up to ~2 minutes. If the webhook was missed, this endpoint asks the gateway directly (`gateway.verify(...)`).

Make activation safe to run twice: at the top of your activation code, `if (payment.status === 'completed') return;`. Gateways sometimes send the same webhook twice.

Webhooks need a public URL. `localhost` can't receive them. During development, use a tunnel tool like ngrok to expose your local server, or rely on the polling backup with the simulator.

Step 5: Subscription lifecycle (about 3 hours)

Activation (runs once after confirmed payment):

```
const endDate = new Date();
endDate.setDate(endDate.getDate() + 30);   // 30 days

await Subscription.create({ userId, planName, amount, status: 'active',
                            startDate: new Date(), endDate });
await User.findByIdAndUpdate(userId, { plan: planName, planExpiresAt: endDate });
```

If the user already has an active plan, **extend** from their current end date rather than from today, so they don't lose paid days.

Expiry (the part currently missing, which is why users stay premium forever):

Create `src/services/subscriptionService.ts` with a function that finds users whose `planExpiresAt` is in the past and sets them back to `free`. Run it on a schedule using the `node-cron` package:

```
import cron from 'node-cron';
cron.schedule('0 * * * *', expireSubscriptions);   // every hour
```

Second safety net: inside your permission check (Step 6), also compare `planExpiresAt` with the current time. Then access stops at the exact moment of expiry even if the hourly job hasn't run yet.

Optional, but useful: a **3-day grace period**, and notifications 7, 3 and 1 day before expiry.

Step 6: Lock features on the backend (about 2 hours)

Create `src/middleware/requireFeature.ts`:

```
import { hasFeature } from '../config/plans';

export const requireFeature = (feature: string) =>
  (req, res, next) => {
    const user = req.user;                        // set by your login middleware
    const expired = user.planExpiresAt && user.planExpiresAt < new Date();
    const plan = expired ? 'free' : user.plan;

    if (!hasFeature(plan, feature)) {
      return res.status(403).json({ error: 'upgrade_required', feature });
    }
    next();
  };
```

Use it on routes:

```
router.get('/analytics/advanced', auth, requireFeature('advanced_analytics'), getAdvancedAnalytics
    );
router.post('/promotions', auth, requireFeature('promotions'), createPromotion);
```

Upload limits are a count check, not a feature check. In the upload route:

```
const count = await Song.countDocuments({ artistId: user._id });
if (count >= PLANS[plan].maxUploads) {
  return res.status(403).json({ error: 'upload_limit_reached' });
}
```

Premium songs: add `isPremium` to your Song model. When a free user requests a premium song's audio, the server returns a **30-second preview** (or an upgrade error). Do not just hide the Play button, because the audio file URL itself must be protected. Don't expose the full file URL for premium songs to free users.

Test it with a free account by calling the API directly (use Postman or Thunder Client). You should get `403`.

Step 7: Frontend upgrade experience (about 3 hours)

Now that the backend is safe, the UI is just a convenience layer.

1. **Read the plan from the server**, not from local state. After login and after payment, call `GET /api/users/me`.
2. **Locked features:** show them visibly but locked (crown icon crown icon, "Upgrade to unlock").
3. **Handle the `403 upgrade_required` response** globally in your API helper: when it appears, open the upgrade screen.
4. **Payment modal:** after the user submits their phone number, show "Check your phone and approve the payment", poll the status (Step 4d), then show success or failure.
5. **"My Subscription" card** in Fan/Artist settings: plan name, expiry date, days remaining, Renew button.
6. **Payment history** page listing date, amount, purpose and status.

Step 8: Tips, wallet and commission (about 4 hours)

Never store an artist's balance as a single number you edit. If it ever goes wrong, you can't tell why. Use a **ledger**: a list of every movement, where the balance is the sum.

Create a `WalletTransaction` model:

Field	Example
`userId`	the artist
`type`	`tip_received`, `commission`, `withdrawal`, `refund`
`amount`	positive for money in, negative for money out
`relatedPaymentId`	links to the Payment
`createdAt`	timestamp

When a tip of 1,000 XAF is confirmed (same webhook flow as Step 4):

- Add `+900` `tip_received` for the artist
- The platform keeps `100` as commission (store the commission rate in `.env` as `PLATFORM_COMMISSION=0.10`)

Artist balance = sum of that artist's ledger entries.

Withdrawals:

1. Artist requests an amount. Check: amount ≤ balance, and above a minimum (e.g. 1,000 XAF).
2. Create a `Withdrawal` with `status: 'pending'` and **immediately** add a negative ledger entry, so they can't request the same money twice.
3. Admin reviews and approves (manual approval is wise at first, to catch fraud).
4. Backend calls `gateway.disburse(...)`. On success mark `completed`; on failure mark `failed` and add a refund entry.

Step 9: Promotion credits (about 2 hours)

Add `promotionCredits: Number` to the User. Buying credits is just another payment with `purpose: 'credits'`; on confirmation, add the credits.

Spending: `POST /api/promotions` checks the artist has enough credits and has the `promotions` feature, then deducts credits and sets `Song.boostedUntil` (e.g. now + 7 days). Your discovery feed puts songs where `boostedUntil > now` first.

Step 10: Admin revenue dashboard (about 2 hours)

Create endpoints (admin-only) using MongoDB aggregation:

- `GET /api/stats/revenue`: total completed payments by day/week/month
- `GET /api/stats/subscriptions`: active subscriptions per plan
- Failed payments list, and a manual "gift premium" action

Show these in your Admin Dashboard as number cards, a simple line chart (the `recharts` library works well) and a recent transactions table.

Step 11: Security and hardening (about 2 hours)

Don't skip this section, because it involves real money.

- **Rate limit** payment routes with `express-rate-limit` (e.g. 5 attempts per minute per user).
- **Validate** amounts and phone numbers on the server (Cameroon MoMo numbers: 9 digits starting with 6).
- **Verify webhook signatures** so nobody can fake "payment successful" by calling your URL.
- **Never put secrets in code or the frontend.** API keys go in `.env`, and `.env` goes in `.gitignore`.
- **Log** every payment event (initiated, confirmed, failed) so you can investigate disputes.
- **Test these scenarios:** failed payment, user closes the app mid-payment, webhook arrives twice, plan expires, user pays while already subscribed.

5.4 Going Live With Real MoMo

You build this last, after everything works with the simulator.

1. **Choose an aggregator.** Campay and Fapshi are usually faster to start with than a direct MTN integration (which requires heavier business onboarding). Both give sandbox credentials. Check their current fees and requirements on their websites before choosing, as these change.
2. **Create a sandbox account** and get your API key and secret.
3. **Write** `campayGateway.ts` (or the equivalent) that `implements PaymentGateway`. Read the provider's documentation for the "collect" and "transaction status" endpoints and map their responses into your `InitiateResult` and `VerifyResult`.
4. **Register it** in `gatewayFactory.ts`, then set `PAYMENT_GATEWAY=campay` and add the credentials to `.env`.
5. **Deploy your backend** to a public HTTPS URL (Render, Railway or a VPS) and give the provider your webhook URL.
6. **Test with sandbox numbers**, trying success, failure and timeout cases.

7. **Apply for production access.** Expect to submit business and ID documents (KYC).
8. **Go live gently:** switch the env variable, test with your own phone and a small amount (e.g. 100 XAF), then open it to users.

Your frontend needs no changes for this switch, which is the point of Step 3.

5.5 Master Checklist

- [ ] 1. User model: `role`, `plan`, `planExpiresAt` and data migration
- [ ] 2. `plans.ts` with entitlements
- [ ] 3. Gateway interface, simulated gateway, factory
- [ ] 4. Pending payments, idempotency, server-side prices, webhook and polling
- [ ] 5. Subscription activation, extension, hourly expiry job
- [ ] 6. `requireFeature` middleware, upload limits, premium song protection
- [ ] 7. Upgrade screens, subscription card, payment history
- [ ] 8. Wallet ledger, commission, withdrawals
- [ ] 9. Promotion credits and boosting
- [ ] 10. Admin revenue dashboard
- [ ] 11. Rate limiting, signature checks, logging, edge-case tests
- [ ] 12. Real gateway sandbox, deploy, KYC, go live

Realistic timeline: about 25 to 30 hours for a beginner, since learning time is on top of the estimates above. Work in order and test each step before moving on.

5.6 Risks and How This Plan Handles Them

Risk	How this plan handles it
Users bypass premium	Backend `requireFeature` checks (Step 6)
Payment failures	Webhook plus polling backup, retries (Step 4)
Payout errors or fraud	Ledger, manual approval, logs (Steps 8, 11)
Copyright abuse	Your existing moderation and takedown tools
Low adoption	Launch in Cameroon only, keep prices low, adjust after real data

Naming note: file paths in your old plan say `camsound-backend`. Rename them to match AfroRythm when convenient, but it's cosmetic and can wait.

6. Campay Payments Reference

A reusable guide to taking MTN and Orange Mobile Money payments with Campay, in your own apps (AfroRythm) and in WooCommerce stores.

> **How this connects to the rest of the plan.** Section 5.4 gives the short version of going live. This chapter is the full Campay reference. For Campay it replaces the generic gateway steps in 5.4 and adds WooCommerce, so you can reuse it on client projects too. Everything marked **Confirm** is something to check in Campay's own docs before launch (listed in 6.10).

6.1 How Campay Works

Campay is an aggregator. You integrate once and it collects from, and pays out to, both MTN Mobile Money and Orange Money. Your code never talks to MTN or Orange directly, and the same API calls work for both networks. Campay works out the network from the phone number.

The payment flow

1. Your **server** asks Campay for an access token using your app username and password.
2. Your server calls **collect** with the amount, currency (XAF), the customer's phone number and your own reference.
3. Campay sends a payment prompt to the customer's phone. The customer approves it with their PIN.
4. Campay calls your **webhook** with the result. You can also ask for the status yourself (polling).
5. Your server checks the result, marks the order or plan as paid, and the customer gets what they bought.

> **Same rule as Chapter 5.** The browser never decides that a payment succeeded. Only your server does, after Campay confirms. This is true for the AfroRythm backend and for WooCommerce.

Words you will see

Term	Plain meaning
Collect	Money coming in. You ask a customer's phone to approve a payment.
Withdraw	Money going out. You send money from your Campay balance to a phone (artist payouts, refunds).
Reference	Campay's own ID for one transaction. Returned by collect and withdraw.
External reference	Your ID (your Payment ID or WooCommerce order ID), sent with the request so you can match the result later.
Webhook key	A secret from your Campay app. Campay signs every callback with it so you can prove the callback is genuine.
Payment link	A Campay-hosted checkout page. You send the customer there instead of building your own payment form.

6.2 Demo vs Live

Campay runs two completely separate environments. Mixing them up is the most common cause of confusing errors.

	Demo (testing)	Live (real money)
Dashboard	demo.campay.net	campay.net
API base URL	`https://demo.campay.net`	`https://www.campay.net`
Credentials	Its own app username, password and webhook key	A different set, created in the live dashboard
Money	Simulated. Only Campay's test phone numbers succeed.	Real MTN and Orange wallets are charged
A real MoMo number	Declined: "Payment was declined by payer or insufficient funds". This is expected, not a bug.	Works
Use it for	Building, and testing success and failure paths	Launch and real customers

6.3 Setting Up a Demo (Test) Account

1. Create an account at **demo.campay.net**. Confirm your email with the activation link (check spam, or use Resend). Registering an app may be blocked until this is done.
2. Go to **Applications** and click **Register application**. Only the app name is required. Website and logo are optional. Submit.
3. Open the new app and copy its **username**, **password** and **webhook key**. These are demo credentials only.
4. Click **API Docs** in the dashboard. Find the demo test phone numbers there and which ones simulate success and which simulate failure. **Confirm**.
5. Put the credentials in your app (6.6 for AfroRythm, 6.7 for WooCommerce).
6. Make the webhook reachable. Campay must be able to call your server, and localhost is not public. For local development use a tunnel (ngrok or Cloudflare Tunnel), or Local's Live Link for WordPress. Put the public URL in the callback/webhook setting of your Campay app, and update it whenever the tunnel URL changes.
7. Test with the test numbers only. Check the **Transactions** tab in the Campay dashboard and the order or payment status in your own system.
8. Do not click **Go Live** until you are ready to launch.

6.4 Going Live (Account Side)

Do this early, because approval can take time and depends on what Campay asks for.

1. Click **Go Live** in the demo dashboard and complete the verification Campay requests. Expect business or identity details (KYC) and the account that will receive your money. **Confirm** the exact requirements and fees.
2. In the **live** dashboard, register the app again. You get a new username, password and webhook key.
3. Set the live webhook/callback URL to your hosted domain over **HTTPS**. A tunnel URL is not acceptable for live.

4. In your app, switch the environment variables (base URL, username, password, webhook key) to the live values.
5. Make a small real payment with your own MTN number, then your own Orange number. Confirm the order or plan activates and the money shows in your Campay balance.
6. Keep demo and live credentials in separate environment files. Never commit either to Git.

6.5 API Cheat Sheet

Base URL is `https://demo.campay.net` for demo and `https://www.campay.net` for live. Every call except the token call needs the header `Authorization: Token <token>`.

Endpoint	Purpose	Send	You get back
`POST /api/token/`	Log in and get an access token	username, password	token, expires_in
`POST /api/collect/`	Ask a customer to pay	amount, currency (XAF), from (237XXXXXXXXX), description, external_reference	reference, ussd_code, operator
`GET /api/transaction/{reference}/`	Check the status of a payment or payout	nothing	status, amount, currency, operator, operator_reference
`POST /api/withdraw/`	Pay money out to a phone	amount, to (237XXXXXXXXX), description, external_reference	reference, status
`POST /api/get_payment_link/`	Create a hosted checkout link	amount, currency, description, external_reference, redirect_url; optional failure_redirect_url, payment_options (MOMO or CARD), email, first_name, last_name	link, reference
`GET /api/balance/`	Read your app balance	nothing	total_balance, mtn_balance, orange_balance, currency
`GET /api/holder_info/?phone_number=`	Get the name registered to a number	phone_number	full_name

Facts worth remembering

- Phone numbers go to Campay as **237** followed by the 9-digit number (12 digits, for example 237 6XX XXX XXX). Your own users type 9 digits, so add the prefix in code.
- Amounts are whole XAF. No decimals.
- Transaction status is **PENDING** until the customer acts, then **SUCCESSFUL** or **FAILED**. Treat PENDING as "not decided yet", never as failure.
- The webhook delivers: status, reference, amount, currency, operator, code, operator_reference, external_reference, description and **signature**. The fields arrive as query parameters, so read both the query and the body.

- The signature is a token signed with your webhook key (JWT, HS256). Verify it before trusting anything in the callback. **Confirm** the algorithm in the docs.

6.6 Integrating Campay into AfroRythm (Node.js / Express)

This plugs into the adapter pattern from Section 5.3, Step 3. Your routes keep calling `getPaymentGateway()`, and only the factory changes. Match the method names and types below to the `PaymentGateway` interface you already wrote.

6.6.1 Environment variables

```
PAYMENT_GATEWAY=campay
CAMPAY_BASE_URL=https://demo.campay.net     # live: https://www.campay.net
CAMPAY_USERNAME=your_app_username
CAMPAY_PASSWORD=your_app_password
CAMPAY_WEBHOOK_KEY=your_webhook_key
```

6.6.2 The Campay gateway class

Save as `src/services/campayGateway.ts`. It needs Node 18 or newer (built-in `fetch`). It caches the token and retries once if Campay answers 401.

```
const BASE = process.env.CAMPAY_BASE_URL!;
let cached: { token: string; expiresAt: number } | null = null;

async function getToken(): Promise<string> {
  if (cached && cached.expiresAt > Date.now() + 30_000) return cached.token;
  const res = await fetch(`${BASE}/api/token/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: process.env.CAMPAY_USERNAME,
                           password: process.env.CAMPAY_PASSWORD }),
  });
  if (!res.ok) throw new Error(`Campay token error ${res.status}`);
  const data = await res.json();                          // { token, expires_in }
  cached = { token: data.token, expiresAt: Date.now() + data.expires_in * 1000 };
  return data.token;       // Confirm expires_in is seconds. The 401 retry below covers a mistake.
}

async function campayFetch(path: string, init: RequestInit = {}, retry = true): Promise<any> {
  const token = await getToken();
  const res = await fetch(`${BASE}${path}`, { ...init, headers: {
    'Content-Type': 'application/json', Authorization: `Token ${token}` } });
  if (res.status === 401 && retry) { cached = null; return campayFetch(path, init, false); }
  const body = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(`Campay ${path} ${res.status}: ${JSON.stringify(body)}`);
  return body;
}

// users type 6XXXXXXXX (9 digits); Campay wants 237 + those 9 digits
export const toCampayPhone = (p: string) => '237' + p.replace(/\D/g, '').slice(-9);

export class CampayGateway /* implements PaymentGateway */ {
  async initiate(p: { amount: number; phone: string; description: string; paymentId: string }) {
    const r = await campayFetch('/api/collect/', { method: 'POST', body: JSON.stringify({
      amount: String(p.amount), currency: 'XAF', from: toCampayPhone(p.phone),
      description: p.description, external_reference: p.paymentId }) });
    return { gatewayTransactionId: r.reference, ussdCode: r.ussd_code, operator: r.operator };
  }

  async verify(reference: string) {
    const t = await campayFetch(`/api/transaction/${reference}/`);
    return { status: t.status as 'PENDING' | 'SUCCESSFUL' | 'FAILED',
             amount: Number(t.amount), operator: t.operator };
```

```
  }

  async disburse(p: { amount: number; phone: string; description: string; withdrawalId: string }) {
    const r = await campayFetch('/api/withdraw/', { method: 'POST', body: JSON.stringify({
      amount: String(p.amount), to: toCampayPhone(p.phone),
      description: p.description, external_reference: p.withdrawalId }) });
    return { gatewayTransactionId: r.reference, status: r.status };
  }

  balance() { return campayFetch('/api/balance/'); }   // check before approving payouts
}
```

6.6.3 Register it in the factory

```
// src/services/gatewayFactory.ts
case 'campay': return new CampayGateway();
```

In Step 4a, store Campay's `reference` in the Payment's `gatewayTransactionId` field and send your Payment ID as the external reference. Prices still come from `PLANS` on the server, never from the browser.

6.6.4 The webhook

Install `jsonwebtoken` for the signature check. Answer Campay quickly, then do the work. Never activate a plan from the callback alone: re-check the status with Campay and compare the amount with what you recorded.

```
import jwt from 'jsonwebtoken';

router.all('/webhook', async (req, res) => {
  // router mounted at /api/momo, so the public URL is /api/momo/webhook
  const q: any = { ...req.query, ...req.body };       // Campay sends query parameters
  try {
    jwt.verify(String(q.signature), process.env.CAMPAY_WEBHOOK_KEY!, { algorithms: ['HS256'] });
  } catch { return res.status(400).send('bad signature'); }
  res.sendStatus(200);                                // reply fast, then process

  try {
    const payment = await Payment.findOne({ gatewayTransactionId: String(q.reference) });
    if (!payment || payment.status === 'completed') return;        // safe if sent twice
    const t = await getPaymentGateway().verify(payment.gatewayTransactionId);
    if (t.status === 'SUCCESSFUL' && t.amount === payment.amount) await activatePayment(payment);
    else if (t.status === 'FAILED') await markPaymentFailed(payment);
  } catch (e) { console.error('campay webhook error', e); }
});
```

6.6.5 Safety net: polling and reconciliation

Webhooks can be missed (server restart, network drop). Keep the 3-second polling endpoint from Step 4d, and add a job that re-checks stuck payments. It should run the same verify-then-activate logic as the webhook.

```
cron.schedule('*/5 * * * *', async () => {
  const stuck = await Payment.find({ status: 'pending',
                                     createdAt: { $lt: new Date(Date.now() - 3 * 60_000) } });
  for (const p of stuck) { /* verify(p.gatewayTransactionId) -> activatePayment or markPaymentFailed */ }
});
```

6.6.6 Payouts to artists (withdraw)

- Your Campay balance must cover the payout. Call `balance()` first, and never approve a withdrawal larger than the balance for that network (`mtn_balance` or `orange_balance`).

- Before sending, call `holder_info` and show the artist the registered name, so a mistyped number does not send money to a stranger.
- Keep the ledger flow from Step 8: write the negative entry first, call `disburse()`, then mark completed. If it fails, write the refund entry.
- Check Campay's fees and withdrawal limits before setting your minimum payout. **Confirm**.

6.6.7 Alternative: the hosted payment link

If you would rather not build your own phone-number form (for example in a mobile app), create a payment link and send the user to it. The page lets them choose how to pay and returns them to your site.

```
const r = await campayFetch('/api/get_payment_link/', { method: 'POST', body: JSON.stringify({
  amount: '2000', currency: 'XAF', description: 'Artist Starter - 30 days',
  external_reference: payment.id, redirect_url: 'https://yourdomain.com/pay/done',
  failure_redirect_url: 'https://yourdomain.com/pay/failed', payment_options: 'MOMO' }) });
// r = { link, reference }  -> redirect the user to r.link, still confirm via webhook
```

6.6.8 MTN and Orange

Nothing changes in code. Campay detects the network from the number and reports it in the `operator` field, which you can store for analytics (for example, which network your fans use most). Always test one MTN and one Orange number before launch.

6.7 Using Campay in WooCommerce

6.7.1 The official plugin (fastest route)

1. Install and activate **WooCommerce**, then install **Campay Woocommerce Payment Gateway** from the plugin directory. The plugin only activates when WooCommerce is active. Set the store currency to **XAF**.
2. Go to WooCommerce, Settings, Payments, and enable **CamPay Payment Gateway**.
3. Enter the app **username**, **password** and **webhook key** (demo values while testing, live values on the real site).
4. At checkout the customer picks CamPay, types a 9-digit MTN or Orange number, and approves the prompt on their phone.
5. The order should move out of pending after payment. Check WooCommerce, Orders, and the Campay Transactions tab.

The plugin's directory listing says it was last tested on WordPress 6.5 and WooCommerce 8.7.0. It usually keeps working on newer versions, but always run a full test checkout after updating WordPress, WooCommerce or the plugin.

6.7.2 Testing on a local site (Local / wplocal)

- Use **demo** credentials and Campay's test numbers. A real number is declined in demo. That is normal.
- Order stays pending although Campay shows success? The webhook cannot reach your computer. Turn on Local's **Live Link** (or use ngrok), then use that public address for the store and for the webhook URL in Campay.
- The Live Link address changes every time you disable and re-enable it. Update Campay each time.
- Debugging: set `WP_DEBUG` and `WP_DEBUG_LOG` to true in wp-config.php and read `wp-content/debug.log`. Also see WooCommerce, Status, Logs.

6.7.3 Going live on the hosted site

1. Host the site on a real domain with **HTTPS** and migrate it from Local. Confirm WooCommerce and the Campay plugin are active.
2. Complete **Go Live** with Campay (6.4) and create the live app.
3. Replace the demo username, password and webhook key in the plugin settings with the live ones. Set the live webhook URL in Campay.
4. Make sure no security or firewall plugin blocks the webhook route.
5. Place a small real order and pay with your own MTN number, then your own Orange number. Confirm the order status updates, then refund or hide the test product.

6.7.4 When to build a custom gateway

Use your own gateway class if the plugin breaks on a new WooCommerce version, or when a client needs something it lacks (custom order statuses, saved phone numbers, refunds through withdraw). This skeleton shows the three parts that matter. It uses the same Campay calls as 6.6, so keep the token cached with `set_transient()`.

```
class WC_Gateway_Campay_Custom extends WC_Payment_Gateway {
  public function __construct() {
    $this->id = 'campay_custom';  $this->method_title = 'Campay (custom)';
    $this->has_fields = true;     $this->supports = ['products'];
    $this->init_form_fields();    $this->init_settings();
    add_action('woocommerce_update_options_payment_gateways_' . $this->id, [$this, 'process_admin_options']);
    add_action('woocommerce_api_' . $this->id, [$this, 'webhook']);   // /?wc-api=campay_custom
  }
  public function payment_fields() { echo '<input name="campay_phone" placeholder="6XXXXXXXX">'; }

  public function process_payment($order_id) {
    $order = wc_get_order($order_id);
    $phone = '237' . substr(preg_replace('/\D/', '', $_POST['campay_phone'] ?? ''), -9);
    // POST {base}/api/collect/ with amount (whole XAF), currency XAF, from $phone,
    // description, external_reference = $order_id, header "Authorization: Token ..."
    // $ref = reference from the response; on error: wc_add_notice(...) and return failure
    $order->update_meta_data('_campay_ref', $ref);
    $order->update_status('on-hold', 'Waiting for MoMo approval');
    $order->save();
    WC()->cart->empty_cart();
    return ['result' => 'success', 'redirect' => $this->get_return_url($order)];
  }

  public function webhook() {
    $sig = sanitize_text_field($_REQUEST['signature'] ?? '');
    if (!campay_jwt_valid($sig, $this->get_option('webhook_key'))) { status_header(400); exit; }
    // find the order by external_reference, GET /api/transaction/{reference}/ to confirm,
    // then if SUCCESSFUL and the amount matches: $order->payment_complete($reference);
    status_header(200); exit;
  }
}

function campay_jwt_valid(string $jwt, string $key): bool {       // HS256 check, no library needed
  $p = explode('.', $jwt);  if (count($p) !== 3) return false;
  $sig = rtrim(strtr(base64_encode(hash_hmac('sha256', "$p[0].$p[1]", $key, true)), '+/', '-_'), '=');
  return hash_equals($sig, $p[2]);
}
```

6.8 Go-Live Checklist (Any Project)

```
[ ] 1. Demo app registered; full checkout tested with Campay test numbers
[ ] 2. Success, failure, and "customer never approves" cases all behave correctly
[ ] 3. Webhook verifies the signature, re-checks the status, and is safe if sent twice
[ ] 4. Polling and reconciliation catch payments whose webhook was missed
[ ] 5. Site or backend deployed on a real domain with HTTPS
[ ] 6. Go Live submitted; Campay verification (KYC) approved
[ ] 7. Live app created; live username, password and webhook key stored in env files
[ ] 8. Live webhook URL set in Campay; credentials are not in Git or the frontend
[ ] 9. Small real payment tested on MTN and on Orange
[ ] 10. Small payout tested (AfroRythm) and balance reconciled in the dashboard
[ ] 11. Logging on: every initiate, callback, verify, activate and payout event
```

6.9 Troubleshooting

Symptom	Likely cause and fix
"Payment was declined by payer or insufficient funds" on a real number	You are on the demo environment. Use Campay's test numbers, or go live.
Campay shows SUCCESSFUL but your order or plan stays pending	Webhook not reaching you (localhost, wrong URL, no HTTPS, firewall). Use a tunnel or Live Link in testing, check the URL, and rely on polling as a backup.
401 or "invalid token"	Wrong environment (demo credentials against the live URL or the reverse), expired token, or wrong password. Refetch the token.
Invalid phone or request rejected	Number not in 237 + 9 digits format, or an amount with decimals.
Same payment activated twice	Missing idempotency. Return early if the payment is already completed, and look it up by Campay reference.
Customer approved but your app shows failure	You stopped polling too early or treated PENDING as failed. Keep the status as pending and let reconciliation decide.
Payout fails	Insufficient balance on that network, invalid number, or a limit. Check balance() and the error body, then write the refund ledger entry.
Works locally, breaks after hosting	Still using demo credentials, old tunnel URL in Campay, HTTP instead of HTTPS, or caching/security plugin blocking the webhook route.

6.10 Confirm in Campay's Docs Before Launch

These details were not verifiable from the sources used to write this chapter. Check them in the API Docs inside your Campay dashboard and fill them in here. Campay's documentation is the source of truth: if anything in this chapter disagrees with it, trust the docs (documenter.getpostman.com/view/2391374/T1LV8PVA) and update this page.

- Demo test phone numbers and which outcome each one simulates.
- Exact webhook signature format and algorithm, and whether Campay sends GET or POST.
- Go Live requirements: documents needed, approval time, and any setup fees.
- Transaction fees for collect and withdraw, on both MTN and Orange.
- Minimum and maximum amounts per collect and withdraw, and daily limits.
- Token lifetime (the `expires_in` unit) and rate limits.
- How long a payment prompt stays valid before it turns FAILED.